

## ■ IRON LINKS GYM

Tarea #3 — Reto Final

# Memoria Técnica

---

|                     |                                       |
|---------------------|---------------------------------------|
| <b>Alumno:</b>      | Dan Bolocan                           |
| <b>Módulo:</b>      | Bases de Datos — PL/SQL               |
| <b>Ciclo:</b>       | 1.º DAW · CPIFP Los Enlaces           |
| <b>Caso de uso:</b> | Gimnasio con socios, tarifas y clases |
| <b>Fecha:</b>       | 05/05/2026                            |

## Índice

- |    |                                              |
|----|----------------------------------------------|
| 1. | Visión general del caso de uso               |
| 2. | Modelo Entidad-Relación (E/R)                |
| 3. | Modelo relacional y normalización            |
| 4. | Descripción de tablas y restricciones        |
| 5. | Datos de ejemplo                             |
| 6. | Batería de consultas SQL                     |
| 7. | PL/SQL: procedimientos, funciones y cursores |
| 8. | Triggers                                     |
| 9. | Conclusión                                   |

## 1. Visión general del caso de uso

El caso de uso elegido es **Iron Links Gym**, un gimnasio moderno que necesita gestionar socios, tarifas de abono, contratos, pagos, clases grupales y la disponibilidad de sus instalaciones. La base de datos cubre el ciclo de vida completo del socio: alta, contratación de tarifa, inscripción a clases, seguimiento de asistencia, renovación y baja.

Este dominio resulta especialmente rico para PL/SQL porque combina reglas de negocio que deben aplicarse de forma automática (control de aforo, cálculo de fecha de vencimiento, auditoría RGPD) con operaciones transaccionales complejas (renovación de contratos, inscripciones condicionadas a la vigencia del abono).

### Entidades principales

| Entidad           | Rol en el negocio                                                    |
|-------------------|----------------------------------------------------------------------|
| Sala              | Espacio físico donde se imparten las clases. Tiene aforo máximo.     |
| Monitor           | Profesional que imparte las clases. Tiene especialidad y contacto.   |
| Tarifa            | Plan de abono (mensual, trimestral, anual...) con precio y duración. |
| Socio             | Cliente del gimnasio. Tiene NIF único, estado y fecha de alta.       |
| Contrato          | Vincula un socio con una tarifa en un período de tiempo concreto.    |
| Clase             | Sesión grupal con sala, monitor, horario y límite de plazas.         |
| Inscripcion_clase | Relación N:M entre Socio y Clase. Registra fecha y asistencia.       |
| Pago_cuota        | Registro de cada cobro asociado a un contrato.                       |
| Auditoria_socios  | Historial de cambios de estado en la entidad Socio (RGPD).           |

## 2. Modelo Entidad-Relación (E/R)

A continuación se describe el diagrama E/R en notación de texto. Las relaciones se representan indicando cardinalidad, entidades participantes y atributos de la relación.

| Relación    | Entidades             | Cardinalidad | Atributos propios                                                    |
|-------------|-----------------------|--------------|----------------------------------------------------------------------|
| TIENE       | SALA – CLASE          | 1 : N        | —                                                                    |
| IMPARTE     | MONITOR – CLASE       | 1 : N        | —                                                                    |
| CONTRATA    | SOCIO – TARIFA        | N : M        | fecha_inicio, fecha_fin, importe_pag, estado (resuelta por CONTRATO) |
| SE_INSCRIBE | SOCIO – CLASE         | N : M        | fecha_insc, asistio (resuelta por INSCRIPCION_CLASE)                 |
| GENERA      | CONTRATO – PAGO_CUOTA | 1 : N        | fecha_pago, importe, metodo_pago                                     |
| AUDITA      | SOCIO – AUDITORIA     | 1 : N        | accion, campo, valor_antes, valor_despues                            |

### Justificación de las relaciones N:M

**SOCIO – TARIFA (CONTRATO):** un socio puede contratar varias tarifas a lo largo del tiempo (renovaciones) y una tarifa puede ser contratada por muchos socios. La tabla intermedia CONTRATO añade los atributos temporales y económicos de cada vinculación.

**SOCIO – CLASE (INSCRIPCION\_CLASE):** un socio puede inscribirse en muchas clases y una clase puede tener muchos socios. La tabla intermedia registra cuándo se inscribió y si finalmente asistió, datos que no pertenecen ni al socio ni a la clase por separado.

### 3. Modelo relacional y normalización

Todas las tablas se encuentran en **3.ª Forma Normal (3FN)**. A continuación se indica el esquema relacional y la justificación de las claves primarias elegidas.

| Tabla             | Clave primaria     | Justificación                                                                                       |
|-------------------|--------------------|-----------------------------------------------------------------------------------------------------|
| SALA              | PK: id_sala        | Clave sintética porque el nombre de la sala puede cambiar.                                          |
| MONITOR           | PK: id_monitor     | NIF no almacenado; email y teléfono son UNIQUE para identificación natural.                         |
| TARIFA            | PK: id_tarifa      | El nombre puede modificarse; la PK no debe cambiar para conservar FK en contratos.                  |
| SOCIO             | PK: id_socio       | NIF marcado UNIQUE como clave natural alternativa.                                                  |
| CONTRATO          | PK: id_contrato    | Un socio puede tener múltiples contratos; la PK distingue cada período.                             |
| CLASE             | PK: id_clase       | Nombre+día+hora no es suficientemente estable como PK natural.                                      |
| INSCRIPCION_CLASE | PK: id_inscripcion | Constraint UNIQUE(id_socio, id_clase) evita duplicados; PK sintética para FK en auditorías futuras. |
| PAGO_CUOTA        | PK: id_pago        | Cada pago es un evento independiente incluso del mismo contrato.                                    |
| AUDITORIA_SOCIOS  | PK: id_auditoria   | Tabla de solo inserción; nunca se borran registros de auditoría.                                    |

#### Verificación de Formas Normales

**1FN:** todos los atributos son atómicos. No hay grupos repetidos ni arrays.

**2FN:** no hay dependencias parciales. En INSCRIPCION\_CLASE, los atributos fecha\_insc y asistio dependen de la clave completa (id\_socio, id\_clase), no de una parte de ella.

**3FN:** no hay dependencias transitivas. Por ejemplo, el precio de la tarifa está en TARIFA, no en CONTRATO (donde se copia como importe\_pag para preservar el valor histórico sin depender transitivamente de TARIFA).

## 4. Descripción de tablas y restricciones

**Tabla: SALA**

| Columna     | Tipo          | Restricción   | Notas          |
|-------------|---------------|---------------|----------------|
| id_sala     | NUMBER        | PK (seq_sala) |                |
| nombre      | VARCHAR2(60)  | NOT NULL      |                |
| aforo_max   | NUMBER(3)     | NOT NULL      | CHECK (1..200) |
| descripcion | VARCHAR2(200) |               |                |

**Tabla: MONITOR**

| Columna      | Tipo          | Restricción | Notas           |
|--------------|---------------|-------------|-----------------|
| id_monitor   | NUMBER        | PK          |                 |
| nombre       | VARCHAR2(80)  | NOT NULL    |                 |
| apellidos    | VARCHAR2(100) | NOT NULL    |                 |
| especialidad | VARCHAR2(60)  |             |                 |
| telefono     | VARCHAR2(15)  | UNIQUE      |                 |
| email        | VARCHAR2(100) | UNIQUE      |                 |
| fecha_alta   | DATE          | NOT NULL    | DEFAULT SYSDATE |

**Tabla: TARIFA**

| Columna       | Tipo          | Restricción | Notas              |
|---------------|---------------|-------------|--------------------|
| id_tarifa     | NUMBER        | PK          |                    |
| nombre        | VARCHAR2(60)  | NOT NULL    |                    |
| duracion_dias | NUMBER(4)     | NOT NULL    | CHECK > 0          |
| precio        | NUMBER(8,2)   | NOT NULL    | CHECK > 0          |
| descripcion   | VARCHAR2(200) |             |                    |
| activa        | CHAR(1)       | NOT NULL    | CHECK IN ('S','N') |

**Tabla: SOCIO**

| Columna  | Tipo         | Restricción     | Notas |
|----------|--------------|-----------------|-------|
| id_socio | NUMBER       | PK              |       |
| nif      | VARCHAR2(10) | NOT NULL UNIQUE |       |
| nombre   | VARCHAR2(80) | NOT NULL        |       |

| Columna    | Tipo          | Restricción | Notas                                     |
|------------|---------------|-------------|-------------------------------------------|
| apellidos  | VARCHAR2(100) | NOT NULL    |                                           |
| fecha_nac  | DATE          | NOT NULL    |                                           |
| telefono   | VARCHAR2(15)  |             |                                           |
| email      | VARCHAR2(100) | UNIQUE      |                                           |
| fecha_alta | DATE          | NOT NULL    | DEFAULT SYSDATE                           |
| estado     | VARCHAR2(10)  | NOT NULL    | CHECK IN ('ACTIVO', 'BAJA', 'SUSPENDIDO') |

### Tabla: CONTRATO

| Columna      | Tipo         | Restricción | Notas                                       |
|--------------|--------------|-------------|---------------------------------------------|
| id_contrato  | NUMBER       | PK          |                                             |
| id_socio     | NUMBER       | FK SOCIO    | NOT NULL                                    |
| id_tarifa    | NUMBER       | FK TARIFA   | NOT NULL                                    |
| fecha_inicio | DATE         | NOT NULL    |                                             |
| fecha_fin    | DATE         |             | Calculada por trigger                       |
| importe_pag  | NUMBER(8,2)  |             |                                             |
| estado       | VARCHAR2(10) | NOT NULL    | CHECK IN ('ACTIVO', 'VENCIDO', 'CANCELADO') |

### Tabla: CLASE

| Columna       | Tipo         | Restricción | Notas                  |
|---------------|--------------|-------------|------------------------|
| id_clase      | NUMBER       | PK          |                        |
| nombre        | VARCHAR2(80) | NOT NULL    |                        |
| id_sala       | NUMBER       | FK SALA     | NOT NULL               |
| id_monitor    | NUMBER       | FK MONITOR  | NOT NULL               |
| dia_semana    | VARCHAR2(10) | NOT NULL    | CHECK (LUNES..DOMINGO) |
| hora_inicio   | VARCHAR2(5)  | NOT NULL    |                        |
| duracion_min  | NUMBER(3)    | NOT NULL    | CHECK (15..240)        |
| plazas_max    | NUMBER(3)    | NOT NULL    | CHECK > 0              |
| plazas_libres | NUMBER(3)    | NOT NULL    | CHECK >= 0             |
| activa        | CHAR(1)      | NOT NULL    | CHECK IN ('S', 'N')    |

**Tabla: INSCRIPCION\_CLASE**

| Columna        | Tipo    | Restricción | Notas                                          |
|----------------|---------|-------------|------------------------------------------------|
| id_inscripcion | NUMBER  | PK          |                                                |
| id_socio       | NUMBER  | FK SOCIO    | NOT NULL                                       |
| id_clase       | NUMBER  | FK CLASE    | NOT NULL                                       |
| fecha_insc     | DATE    | NOT NULL    | DEFAULT SYSDATE                                |
| asistio        | CHAR(1) | NOT NULL    | CHECK IN ('P','S','N');<br>UNIQUE(socio,clase) |

**Tabla: PAGO\_CUOTA**

| Columna       | Tipo          | Restricción | Notas                                                            |
|---------------|---------------|-------------|------------------------------------------------------------------|
| id_pago       | NUMBER        | PK          |                                                                  |
| id_contrato   | NUMBER        | FK CONTRATO | NOT NULL                                                         |
| fecha_pago    | DATE          | NOT NULL    | DEFAULT SYSDATE                                                  |
| importe       | NUMBER(8,2)   | NOT NULL    | CHECK > 0                                                        |
| metodo_pago   | VARCHAR2(20)  | NOT NULL    | CHECK IN (DOMICILIACION,<br>TARJETA, EFECTIVO,<br>TRANSFERENCIA) |
| observaciones | VARCHAR2(200) |             |                                                                  |

**Tabla: AUDITORIA\_SOCIOS**

| Columna       | Tipo          | Restricción | Notas                                   |
|---------------|---------------|-------------|-----------------------------------------|
| id_auditoria  | NUMBER        | PK          |                                         |
| id_socio      | NUMBER        | NOT NULL    | Sin FK (datos históricos inmutables)    |
| accion        | VARCHAR2(10)  | NOT NULL    | CHECK IN ('INSERT', 'UPDATE', 'DELETE') |
| campo         | VARCHAR2(60)  |             |                                         |
| valor_antes   | VARCHAR2(200) |             |                                         |
| valor_despues | VARCHAR2(200) |             |                                         |
| usuario_bd    | VARCHAR2(60)  |             | DEFAULT USER                            |
| fecha_cambio  | DATE          | NOT NULL    | DEFAULT SYSDATE                         |

## 5. Datos de ejemplo (insertar.sql)

Se han insertado datos realistas que permiten probar todos los casos de las consultas y de la lógica PL/SQL. El volumen por tabla es:

| Tabla             | Filas | Observaciones                                                     |
|-------------------|-------|-------------------------------------------------------------------|
| SALA              | 6     | Cardio, Spinning, Musculación, 2 Polivalentes, Piscina            |
| MONITOR           | 6     | Especialidades variadas: yoga, spinning, natación, kickboxing...  |
| TARIFA            | 8     | Mensual/Trimestral/Anual × Básica/Completa + Pase Día + Jubilados |
| SOCIO             | 15    | Incluye estados ACTIVO, BAJA y SUSPENDIDO para probar filtros     |
| CONTRATO          | 16    | Incluye estados ACTIVO, VENCIDO; renovación de un mismo socio     |
| CLASE             | 11    | Distribuidas en distintos días y salas                            |
| INSCRIPCION_CLASE | 20    | Mezcla de asistencias P, S y N para probar agregaciones           |
| PAGO_CUOTA        | 16    | Un pago por contrato + uno de renovación                          |

Los datos cubren casos límite relevantes: socios sin inscripciones (para la consulta 5), clases con pocas plazas libres (consulta 6), contratos de distintos tipos para las estadísticas de facturación (consultas 3 y 4), y socios con contratos cercanos al vencimiento (consulta 10 y cursor de alertas).

## 6. Batería de consultas SQL (consultas.sql)

Se han diseñado 10 consultas que cubren los requisitos del proyecto. Todas incluyen comentario de negocio en el fichero .sql.

| N.º | Técnica SQL                                   | Uso de negocio                                                                      | Clave del código                                                                                 |
|-----|-----------------------------------------------|-------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| 1   | JOIN múltiple (CLASE + SALA + MONITOR)        | Listado de clases activas con sala, monitor y plazas libres. Panel de recepción.    | <pre>SELECT ... FROM clase JOIN sala ... JOIN monitor ... WHERE activa='S'</pre>                 |
| 2   | GROUP BY + COUNT + LEFT JOIN                  | Número de contratos activos por tarifa. El gerente ve qué planes son más populares. | <pre>SELECT t.nombre, COUNT(c.id_contrato) ... GROUP BY t.id_tarifa ...</pre>                    |
| 3   | GROUP BY + HAVING + SUM (JOIN triple)         | Socios que han gastado más de 200 €. Identifica clientes de alto valor.             | <pre>... SUM(p.importe) ... HAVING SUM(p.importe) &gt; 200</pre>                                 |
| 4   | GROUP BY + MIN/MAX/AVG (pago)                 | Estadísticas de facturación por método de pago. Análisis para contabilidad.         | <pre>... MIN, MAX, AVG, SUM ... GROUP BY p.metodo_pago</pre>                                     |
| 5   | Subconsulta EXISTS / NOT EXISTS               | Socios con contrato activo sin ninguna inscripción. Candidatos a venta cruzada.     | <pre>WHERE EXISTS (contrato activo) AND NOT EXISTS (inscripción)</pre>                           |
| 6   | JOIN + expresión derivada + filtro porcentaje | Clases con menos del 30 % de plazas libres. Alerta de aforo.                        | <pre>WHERE (plazas_libres / plazas_max) &lt; 0.30</pre>                                          |
| 7   | GROUP BY + HAVING (carga monitor)             | Monitores con más de una clase activa. Control de carga de trabajo.                 | <pre>... COUNT(c.id_clase) ... HAVING COUNT &gt; 1</pre>                                         |
| 8   | Subconsulta correlacionada (MAX contrato)     | Socios en tarifa Básica con más de 1 año. Candidatos a upgrade.                     | <pre>WHERE id_contrato = (SELECT MAX(id_contrato) FROM contrato WHERE id_socio=s.id_socio)</pre> |
| 9   | JOIN triple + CASE dentro de SUM              | Ranking de clases por asistencias confirmadas. Mide popularidad real.               | <pre>SUM(CASE WHEN asistio='S' THEN 1 ELSE 0 END)</pre>                                          |
| 10  | Vista en línea (subconsulta en FROM) + RANK() | Contratos que vencen en 30 días. Recepción llama para ofrecer renovación.           | <pre>RANK() OVER (PARTITION BY id_socio ORDER BY fecha_fin DESC)</pre>                           |

## 7. PL/SQL: procedimientos, funciones y cursores (plsql.sql)

### 7.1 Procedimiento: inscribir\_socio\_en\_clase

**Parámetros:** p\_id\_socio IN, p\_id\_clase IN, p\_resultado OUT VARCHAR2

**Utilidad de negocio:** gestiona la inscripción de un socio a una clase de forma segura. Antes de insertar verifica: (1) que el socio tiene contrato activo y vigente, (2) que la clase existe y está activa, (3) que hay plazas disponibles, (4) que el socio no estaba ya inscrito. En cada comprobación devuelve un mensaje descriptivo en p\_resultado. Si todo es correcto, inserta en INSCRIPCION\_CLASE y hace COMMIT. El trigger trg\_plazas\_inscripcion descuenta la plaza automáticamente.

**Estructuras PL/SQL usadas:** IF/RETURN, EXCEPTION WHEN NO\_DATA\_FOUND, COMMIT, ROLLBACK.

### 7.2 Procedimiento: renovar\_contrato

**Parámetros:** p\_id\_socio IN, p\_id\_tarifa IN, p\_metodo\_pago IN, p\_id\_contrato\_n OUT, p\_resultado OUT

**Utilidad de negocio:** cuando un abono vence, recepción puede llamar a este procedimiento para: (1) verificar el estado del socio, (2) cerrar los contratos anteriores activos (UPDATE estado='VENCIDO'), (3) crear el nuevo contrato (el trigger calcula la fecha\_fin), (4) registrar el pago correspondiente, (5) reactivar el socio si estaba SUSPENDIDO. Devuelve el id del nuevo contrato y un mensaje de confirmación.

**Estructuras PL/SQL usadas:** múltiples UPDATE/INSERT, manejo de excepciones, COMMIT/ROLLBACK.

### 7.3 Función: calcular\_ingresos\_mes

**Signatura:** calcular\_ingresos\_mes(p\_anio NUMBER, p\_mes NUMBER) RETURN NUMBER

**Utilidad de negocio:** devuelve el total recaudado en un mes y año dados consultando la tabla PAGO\_CUOTA con EXTRACT. Útil para informes mensuales de dirección y para el bloque del cursor (muestra los últimos 3 meses con un WHILE).

### 7.4 Función: plazas\_ocupadas\_clase

**Signatura:** plazas\_ocupadas\_clase(p\_id\_clase NUMBER) RETURN NUMBER

**Utilidad de negocio:** cuenta las inscripciones con estado 'P' (pendiente) o 'S' (asistido), que son las que ocupan plaza efectivamente. Devuelve -1 si la clase no existe. Complementa el campo plazas\_libres de CLASE para verificaciones puntuales.

### 7.5 Cursor explícito: alertas de renovación

**Nombre del cursor:** cur\_vencimientos. Selecciona contratos activos con fecha\_fin entre SYSDATE y SYSDATE+15 (próximos 15 días). Para cada fila aplica lógica condicional (IF: marca como urgente si

quedan 3 días o menos) y construye el mensaje de aviso con CASE. Tras el LOOP principal incluye un bloque WHILE que recorre los 3 meses anteriores e imprime los ingresos de cada uno usando la función calcular\_ingresos\_mes.

**Estructuras usadas:** CURSOR ... IS, OPEN, FETCH, EXIT WHEN %NOTFOUND, CLOSE, LOOP, WHILE, IF, CASE.

## 8. Triggers (triggers.sql)

Se han implementado 6 triggers que automatizan reglas de negocio reales. Cada uno resuelve un problema que no puede cubrirse con restricciones CHECK estándar o que debe ser transparente para cualquier herramienta que acceda a la BD.

### 1. trg\_calcular\_fecha\_fin

**Evento:** BEFORE INSERT ON contrato

**Justificación:** La fecha de fin de un contrato debe ser siempre fecha\_inicio + duracion\_dias de la tarifa elegida. Si dependiera de la aplicación cliente, un descuido del operador dejaría contratos sin fecha\_fin o con fechas incorrectas. El trigger lee la duración de la tarifa y sobrescribe fecha\_fin en :NEW, garantizando coherencia independientemente de cómo se realice el INSERT.

**Clave del código:**

```
Leer duracion_dias de TARIFA y asignar :NEW.fecha_fin := :NEW.fecha_inicio + v_dias
```

### 2. trg\_plazas\_inscripcion

**Evento:** AFTER INSERT ON inscripcion\_clase

**Justificación:** El campo plazas\_libres en CLASE debe decrementarse cada vez que un socio se inscribe. Mantener este contador en la tabla (en lugar de calcular siempre con COUNT) mejora el rendimiento de las consultas de disponibilidad. El trigger además lanza RAISE\_APPLICATION\_ERROR si no hay plazas, evitando inscripciones sobre aforo aunque la comprobación de la aplicación falle.

**Clave del código:**

```
UPDATE clase SET plazas_libres = plazas_libres - 1 WHERE id_clase = :NEW.id_clase
```

### 3. trg\_plazas\_baja

**Evento:** AFTER DELETE ON inscripcion\_clase

**Justificación:** Complemento del trigger anterior. Cuando un socio cancela su inscripción, la plaza debe quedar libre para que otro socio pueda ocuparla. Sin este trigger habría plazas bloqueadas fantasma. El CHECK plazas\_libres < plazas\_max evita superar el aforo original por errores de datos previos.

**Clave del código:**

```
UPDATE clase SET plazas_libres = plazas_libres + 1 WHERE id_clase = :OLD.id_clase AND  
plazas_libres < plazas_max
```

#### 4. trg\_auditoria\_socios

**Evento:** AFTER INSERT OR UPDATE OR DELETE ON socio

**Justificación:** El RGPD exige trazabilidad de los cambios en datos personales. Este trigger registra altas (INSERT), cambios de estado o email (UPDATE) y bajas (DELETE) en la tabla AUDITORIA\_SOCIOS con usuario de BD, fecha exacta y valores anterior y posterior. Al operar a nivel de BD, protege incluso frente a accesos directos por SQL\*Plus o herramientas de administración.

**Clave del código:**

```
INSERT INTO auditoria_socios (...) VALUES (...accion, campo, valor_antes, valor_despues...)
```

#### 5. trg\_validar\_edad\_socio

**Evento:** BEFORE INSERT OR UPDATE ON socio

**Justificación:** El gimnasio no admite socios menores de 16 años sin autorización especial. Una restricción CHECK no puede comparar fecha\_nac con SYSDATE porque SYSDATE es dinámica y CHECK se evalúa con valores constantes. El trigger calcula la edad real con MONTHS\_BETWEEN / 12 y lanza un error si es inferior a 16, con mensaje informativo que incluye la edad calculada.

**Clave del código:**

```
IF FLOOR(MONTHS_BETWEEN(SYSDATE, :NEW.fecha_nac)/12) < 16 THEN  
  RAISE_APPLICATION_ERROR(-20002, ...)
```

#### 6. trg\_no\_solapamiento\_clase

**Evento:** BEFORE INSERT OR UPDATE ON clase

**Justificación:** Un monitor no puede impartir dos clases a la vez, y una sala no puede albergar dos clases simultáneas. Estas restricciones implican comparar la fila nueva con otras filas de la misma tabla, lo que es imposible con CHECK. El trigger realiza dos COUNT(\*) sobre CLASE filtrando por monitor/sala + día + hora, excluyendo la propia fila en caso de UPDATE (NVL(:NEW.id\_clase, -1)).

**Clave del código:**

```
SELECT COUNT(*) INTO v_conflicto_monitor FROM clase WHERE id_monitor=:NEW.id_monitor AND  
dia_semana=:NEW.dia_semana AND hora_inicio=:NEW.hora_inicio AND id_clase !=  
NVL(:NEW.id_clase, -1)
```

## 9. Conclusión

El proyecto Iron Links Gym demuestra cómo una base de datos Oracle bien diseñada puede cubrir el ciclo de vida completo de un negocio real: desde la alta de un socio hasta la renovación de su abono, pasando por la gestión de clases, el control de aforo y el registro de pagos.

Las decisiones de diseño (claves sintéticas, separación de TARIFA y CONTRATO, tabla de auditoría independiente) están motivadas por requisitos de negocio concretos, no por convención. Los triggers y procedimientos implementados resuelven problemas que no podrían resolverse únicamente con restricciones declarativas, lo que justifica el uso de PL/SQL frente a un diseño puramente relacional.

El código SQL y PL/SQL se ha redactado con nombres descriptivos, indentación consistente y comentarios orientados al negocio (no al código), de forma que cualquier persona del equipo pueda mantenerlo sin necesidad de conocer el contexto original del desarrollo.

---

*Dan Bolocan · CPIFP Los Enlaces · 05/05/2026*